# Python语言与系统设计

（文件操作与应用）

# Python Language & System Design

## 第8章 异常处理

## (File Operation & Applications)



合肥工业大学 计算机与信息学院

# 异常处理

## ◆异常处理: try-except语句

- 观察下面这段小程序：

```
num = eval(input("请输入一个整数: "))
print(num**2)
```

当用户输入的不是数字呢？

```
请输入一个整数: 100
10000
>>>
========== RESTART: E
请输入一个整数: NO
Traceback (most recent call last):
  File "E:/a1.py", line 1, in <module>
    num = eval(input("请输入一个整数: "))
  File "<string>", line 1, in <module>
NameError: name 'NO' is not defined
```

# 异常处理

- Python解释器返回了异常信息，同时程序退出




- Python异常信息中最重要的部分是异常类型，它表明了发生异常的原因，也是程序处理异常的依据

# 异常处理

- Python使用try-except语句实现异常处理，基本语法格式：

```
try:
        <语句块1>
except <异常类型>:
        <语句块2>
```

```
try:
    num = eval(input("请输入一个整数: "))
    print(num**2)
except NameError:
    print("输入错误，请输入一个整数!")
```

```
请输入一个整数: NO
输入错误，请输入一个整数!
>>>
```

# 异常处理

- try-except语句可支持多个except语句，语法格式如下：

```
try:
        <语句块1>
except <异常类型1>:
        <语句块2>
….
except <异常类型N>:
        <语句块N+1>
except:
        <语句块N+2>
```

- 最后一个except语句没有指定任何类型，表示它对应的语句块可以处理所有其他异常。此过程与if-elif-else语句类似，是分支结构的一种表达方式

# 异常处理

```
try:
    alp = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
    idx = eval(input("请输入一个整数: "))
    print(alp[idx])
except NameError:
    print("输入错误，请输入一个整数!")
except:
    print("其他错误")
```

```
请输入一个整数： abc
输入错误，请输入一个整数
>>>
==========
请输入一个整数： 100
其他错误
>>>
```

该程序将用户输入的数字作为索引从字符串alp中返回一个字符；

- 当用户输入非整数字符时，except NameError异常被捕获到，提示升用户输入类型错误；
- 当用户输入数字不在01到25之间时，异常被except捕获，程序打印其他错误信息，执行过程和结果如 下：

# 异常处理

- 除了try和except保留字外，异常语句还可与else和finally保留字配合使用，语法格式如下：

```
try:
        <语句块1>
except <异常类型1>:
        <语句块2>
else:
        <语句块3>
finally:
        <语句块4>
```

try语句结束后，执行finally，不管有没有异常

# 异常处理

采用else和finally修改代码如下：

```
try:
    alp = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
    idx = eval(input("请输入一个整数: "))
    print(alp[idx])
except NameError:
    print("输入错误，请输入一个整数!")
else:
    print("没有发生异常")
finally:
    print("程序执行完毕，不知道是否发生了异常")
```

```
========== RESTART:
请输入一个整数: 5
F
没有发生异常
程序执行完毕，不知道是否发生了异常
>>>
========== RESTART:
请输入一个整数: abc
输入错误，请输入一个整数!
程序执行完毕，不知道是否发生了异常
>>>
========== RESTART:
请输入一个整数: 100
程序执行完毕，不知道是否发生了异常
```

# 异常处理

## 上下文管理

使用上下文管理语句with可以自动管理资源，在代码块执行完毕后，自动还原进入该代码之前的现场或上下文

```
with open("d:\\a1.txt",'r') as f:
    for line in f:
        print(line)
```

使用了 with 语句，不管在处理文件过程中是否发生异常，都能保证 with 语句执行完毕后已经关闭了打开的文件句柄

类似如下代码try.....finally ：

```
f = open("d:\\a1.txt",'r')
try:
    for line in f:
        print(line)
finally:
    f.close()
```

# 异常处理

## assert（断言）

assert用于判断一个表达式，在表达式条件为 false 的时候触发异常。我们不知道程序在哪里会出错，与其让它在运行最崩溃，不如在出现错误条件时就崩溃。

assert的异常参数，其实就是在断言表达式后添加字符串信息，用来解释断言并更好的知道是哪里出了问题。格式如下：
assert expression [, arguments]

```
a = -1
assert a<0,"a<0"
print('继续执行')
assert a>0,"a>0"
```

```
继续执行
Traceback (most recent call last):
  File "C:\Users\Administrator\Desktop\temp\B.py", line 4, in <module>
    assert a>0,"a>0"
AssertionError: a>0
>>>
```

# 异常处理

以下实例判断当前系统是否为 Linux，如果不满足条件则直接触发异常，不必执行接下来的代码：

```
import sys
assert ('linux' in sys.platform), "该代码只能在 Linux 下执行“
# 接下来要执行的代码
```

```
Traceback (most recent call last):
  File "D:\temp\a2.py", line 2, in <module>
    assert ('linux' in sys.platform), "该代码只能在 Linux 下执行"
AssertionError: 该代码只能在 Linux 下执行
>>>
```

# 异常处理

函数调用抛出异常：

```
def f(x,y):
    assert y!=0,"除数为0"
    return x/y
print(f(9,2))
print(f(5,0)) #抛出异常
```